DSA Lab Defense Preparation

40 Questions with Model Answers

This document prepares you for a defense of **Lab 1: Sorting, Complexity** in the repository `1`. The questions progress from fundamental concepts to implementation details, code correction, complexity reasoning, benchmarking, debugging, and the three quicksort improvements.

> Use the answers to understand the reasoning, not to memorise sentences. During a defense, explain what the code is doing and why it is correct.

Section A: Foundations

1. What is the purpose of this lab?

Answer: The lab has two main purposes. First, it teaches us to infer the complexity of insertion sort, merge sort, and quicksort from timing data. Second, it asks us to improve quicksort by testing shuffling, median-of-three pivot selection, and insertion sort for small subarrays. We must measure correctness and performance rather than assuming that an improvement is useful.

2. What is a sorting algorithm?

Answer: A sorting algorithm rearranges the values in an array into a chosen order, usually ascending order. For example, it changes `[5, 2, 4]` into `[2, 4, 5]`. Different algorithms produce the same final result but use different amounts of comparison, movement, memory, and recursion.

3. What does `O(n)`, `O(n log n)`, and `O(n²)` mean?

Answer: They describe how an algorithm's work grows as the input size `n` grows. `O(n)` means the work grows proportionally to `n`. `O(n log n)` means the work grows faster than linear but much slower than quadratic. `O(n²)` means the work grows approximately in proportion to the square of `n`. Big-O ignores constant factors and lower-order terms.

4. Why is an algorithm's fastest measured time not automatically its complexity?

Answer: Complexity concerns the growth pattern as the input becomes larger, not the absolute time for one input. An `O(n)` algorithm with a large constant factor can be slower

than an $O(n \log n)$ algorithm for small inputs. We therefore compare several input sizes and examine how the time scales.

5. What is the difference between total time and per-item time?

Answer: Total time is the time required to sort the complete array. Per-item time is total time divided by the input size:

```
Plain Text

per-item time = total time / n
```

Per-item time helps identify complexity. For $O(n)$, it is approximately constant. For $O(n^2)$, it grows roughly in proportion to n. For $O(n \log n)$, it grows slowly, approximately like $\log n$.

6. If the input size increases ten times, what growth should we expect?

Answer: For $O(n)$, total time should increase about ten times. For $O(n^2)$, it should increase about one hundred times because $(10n)^2 = 100n^2$. For $O(n \log n)$, the increase is usually between those values and gradually approaches about 10–13 times for larger n.

7. Does rejecting linear and quadratic behaviour prove that an algorithm is $O(n \log n)$?

Answer: No. Other growth rates are possible, and noisy measurements can mislead us. However, in this lab the main choices are linear, linearithmic, and quadratic. We should also check that $T(n)/(n \log n)$ is approximately stable before concluding that the measured behaviour is consistent with $O(n \log n)$.

8. Why should small timing values be treated carefully?

Answer: Very small measurements can be affected by Java startup, timer precision, garbage collection, operating-system activity, and other fixed costs. Larger inputs usually reveal the algorithm's growth pattern more clearly. `Bench.java` reduces this problem by repeating fast operations and measuring several times.

Section B: Insertion Sort

9. Explain insertion sort intuitively.

Answer: Insertion sort is like sorting playing cards in your hand. It treats the first value as a sorted section, takes the next value, shifts larger sorted values one position to the right, and

inserts the selected value into the gap. It repeats until the sorted section covers the entire array.

10. What is the invariant of insertion sort?

Answer: Before each outer-loop iteration, the part of the array before the current index is sorted. If the current index is `i`, then `array[0..i-1]` is sorted. After inserting `array[i]`, the sorted region grows by one element. When the loop finishes, the entire array is sorted.

11. Explain this pseudocode line: `current = array[i]`.

Answer: It saves the next unsorted value in a temporary variable. Values may be shifted while the algorithm searches for the correct position, so the selected value must be protected. Without `current`, the selected value could be overwritten during shifting.

12. Explain `j = i - 1`.

Answer: The variable `j` starts at the last position of the sorted section. If the selected value is at index `i`, the sorted section ends at index `i-1`. Therefore, `j = i-1` begins the search immediately to the left of the selected value.

13. What does `array[j + 1] = array[j]` do?

Answer: It copies the value at index `j` one position to the right. It is a shift, not a swap. If `array[j]` is larger than `current`, it is too large to remain before `current`, so it moves to `j+1` and creates a logical gap for `current`.

14. Why does insertion sort use `j = j - 1` after shifting?

Answer: After shifting the value at index `j`, the algorithm must inspect the next value to the left. Decreasing `j` moves the search backward through the sorted section. If the new value is smaller than several elements, several shifts occur.

15. Why is the final assignment `array[j + 1] = current`?

Answer: The loop stops either at the beginning of the array or when it finds a value that is not greater than `current`. In both cases, the correct insertion position is immediately after index `j`, which is `j+1`. All larger values have already been shifted right to make that position available.

16. What happens when the next selected value is already larger than the sorted section's last value?

Answer: The `while` condition fails immediately because `array[j] > current` is false. No elements are shifted. The algorithm places `current` at `j+1`, which is its original position. It still performs a comparison, so the time is not zero; it is approximately linear for an already sorted array.

17. Trace insertion sort on `[4, 1, 3, 2]`.

Answer: Treat `[4]` as sorted. Insert `1` by shifting `4`: `[1, 4, 3, 2]`. Insert `3` by shifting `4`: `[1, 3, 4, 2]`. Insert `2` by shifting `4` and `3`: `[1, 2, 3, 4]`. The sorted section grows from one value to four values.

18. What are insertion sort's best and worst cases?

Answer: The best case is an already sorted array. Each value needs approximately one comparison and no shifts, giving `O(n)`. The worst case is usually reverse-sorted input. Each new value must pass almost every earlier value, producing approximately `1 + 2 + ... + n` operations, which is `O(n²)`. Random input is generally also approximately `O(n²)`.

19. What should you say about 95%-sorted data?

Answer: We should rely on the timing evidence and understand how the data was generated. It may be much faster than random input because it has fewer inversions. If the amount of disorder is fixed, the behaviour can be close to linear. If a fixed percentage of elements remains random as `n` grows, the number of disordered elements also grows, so asymptotic behaviour may still be quadratic with a smaller constant factor. The lab asks us to classify the observed timing pattern.

Section C: Merge Sort

20. Is merge sort recursive?

Answer: Yes. The `sort` method recursively divides the current range into a left half and a right half until each range has zero or one elements. However, the `merge` method itself is not recursive; it uses loops and pointers to combine two sorted halves.

21. What is the base case in `Merge.java`?

Answer: The base case is:

```java
if (hi <= lo) return;
```

When `hi` is less than or equal to `lo`, the subarray has zero or one element. Such a subarray is already sorted, so no more splitting is necessary.

22. Explain `int mid = lo + (hi - lo) / 2`.

Answer: It finds the midpoint of the current range `lo..hi`. The range is split into `lo..mid` and `mid+1..hi`. Writing it as `lo + (hi-lo)/2` avoids adding `lo` and `hi` first, which is a safer standard midpoint calculation for large indexes.

23. Why must merge sort sort both halves before merging?

Answer: The merge procedure assumes that each half is already sorted. It compares only the current front value from each half and chooses the smaller one. If either half were unsorted, comparing only the front values would not guarantee a sorted final result.

24. Explain the purpose of the auxiliary array `aux`.

Answer: During merging, the algorithm writes the sorted result back into the original array `a`. It first copies the current range into `aux` so the original left and right values remain available for comparison. `aux` is the safe source, while `a` is the destination.

25. Explain the four conditions in the merge loop.

Answer: The code is:

Java

```
if      (i > mid)          a[k] = aux[j++];
else if (j > hi)           a[k] = aux[i++];
else if (aux[j] < aux[i]) a[k] = aux[j++];
else                       a[k] = aux[i++];
```

If the left half is exhausted, copy from the right. If the right half is exhausted, copy from the left. If both have values, copy the smaller current value. If the values are equal, the `else` branch takes the left value, preserving stability.

26. What does `j++` mean in `a[k] = aux[j++]`?

Answer: It uses the current value at `aux[j]`, assigns it to `a[k]`, and then increases `j` by one. It is equivalent to:

Java

```
a[k] = aux[j];
```

```
j = j + 1;
```

The same explanation applies to `i++`.

27. Merge `[2, 5, 8]` and `[1, 4, 7]` manually.

Answer: Compare `2` and `1`, take `1`. Compare `2` and `4`, take `2`. Compare `5` and `4`, take `4`. Compare `5` and `7`, take `5`. Compare `8` and `7`, take `7`. The right half is exhausted, so copy the remaining `8`. The result is `[1, 2, 4, 5, 7, 8]`.

28. Why is merge sort approximately `O(n log n)`?

Answer: The array is divided in half at each recursive level, giving approximately $\log_2 n$ levels. At each level, all elements are copied and merged once in total, which costs approximately `n` work. Therefore the total work is:

```
n work per level × log n levels = O(n log n)
```

Its overall pattern is not strongly dependent on whether the input is random or already sorted.

Section D: Basic Quicksort

29. What is a pivot in quicksort?

Answer: A pivot is the value around which the current subarray is partitioned. After partitioning, values less than or equal to the pivot are placed on the left and values greater than or equal to it are placed on the right. The pivot is then in its final sorted position.

30. What does partitioning accomplish?

Answer: Partitioning rearranges `a[lo..hi]` so that the pivot ends up at an index `j` satisfying:

```
a[lo..j-1] <= a[j] <= a[j+1..hi]
```

The two sides are not necessarily fully sorted yet. They are only arranged relative to the pivot. Recursive calls sort the two sides.

31. Why does basic quicksort perform badly on sorted input in this lab?

Answer: The implementation chooses the first element as pivot. In an already sorted array, the first element is the smallest value, so one partition is empty and the other contains almost the entire remaining array. This produces highly unbalanced recursion and approximately $O(n^2)$ work.

32. Explain the purpose of `int i = lo; int j = hi + 1;`.

Answer: These are scanning indexes used by the partition procedure. `i` scans from the left toward larger values, and `j` scans from the right toward smaller values. The initial `j = hi + 1` allows the first pre-decrement operation `--j` to inspect `a[hi]`.

33. Why does the partition code use pre-increment and pre-decrement, such as `a[++i]` and `a[--j]`?

Answer: The scan must move before reading the next candidate value. `a[++i]` first increments `i` and then reads `a[i]`. `a[--j]` first decrements `j` and then reads `a[j]`. This matches the intended scanning ranges around the pivot.

34. What is the role of `exchange(a, i, j)`?

Answer: It swaps the values at positions `i` and `j`:

```java
int swap = a[i];
a[i] = a[j];
a[j] = swap;
```

During partitioning, when the left scan finds a value too large and the right scan finds a value too small, exchanging them moves both values toward the correct side.

35. What does the final exchange with the pivot do?

Answer: After the scans meet, the pivot is exchanged with `a[j]`. This places the pivot between the values that belong on its left and the values that belong on its right. The method then returns `j`, the pivot's final index.

36. Why are the recursive calls `sort(a, lo, j-1)` and `sort(a, j+1, hi)`?

Answer: The pivot at index `j` is already in its final position, so it must not be included in either recursive call. The left subarray ends at `j-1`, and the right subarray begins at `j+1`.

Section E: The Three Improvements

37. What does shuffling first improve?

Answer: Shuffling changes the original order of the complete array before quicksort begins. It reduces the chance that a predictable input, such as an already sorted array, repeatedly produces a terrible first-element pivot. It should happen once at the beginning of the public `sort` method, not inside every recursive call.

38. Why can shuffling change the observed complexity of quicksort?

Answer: Without shuffling, sorted input can consistently create unbalanced partitions and quadratic behaviour. With shuffling, the input is likely to behave like a random permutation, so the expected partition balance improves and the observed performance generally moves toward average `O(n log n)`. It does not guarantee balanced partitions on every particular run.

39. What is median-of-three pivot selection?

Answer: It examines the first, middle, and last values of the current subarray and chooses the median value among those three. For example, if the candidates are `1`, `7`, and `4`, the median is `4`. In this implementation, the selected median must be exchanged into `a[lo]` because the existing partition method expects the pivot at the first position.

40. Why use insertion sort for small quicksort subarrays, and how do you choose the cutoff?

Answer: Recursive quicksort and partitioning have overhead. On a very small subarray, insertion sort can be faster because it is simple and avoids additional recursive calls. The cutoff must be found experimentally: test values such as 0, 10, 20, 30, 40, 50, and 60 using the same benchmark conditions, choose the best overall value, and test nearby values to check that performance worsens in both directions. The cutoff improves practical time but, by itself, does not remove the possibility of bad large partitions.

Additional defense follow-ups to practise

The examiner may combine the topics above into follow-up questions. Be ready to explain these points in your own words:

Topic	Short explanation

Original quicksort configuration	`new Quick(false, false, 0)` disables all three improvements.
Fully improved example	`new Quick(true, true, 42)` enables shuffling, median-of-three, and a cutoff of 42.
Why keep the original benchmark	It gives the grader a baseline for comparison.
Correctness before performance	A faster algorithm that produces an incorrect result is not an improvement.
`Bench.java` correctness check	It compares the algorithm's output with Java's `Arrays.sort` result for many generated arrays.
Reproducibility	The benchmark and shuffle use fixed seeds, making tests more repeatable.
Input conditions	`RANDOMNESS = {100, 5, 0}` corresponds to random, 95%-sorted, and sorted descriptions.
Final required files	Improved `Quick.java`, final `output.txt`, and completed `answers.txt` in a group-named ZIP file.

A strong defense explanation

If asked to summarise the whole lab, say:

> "The lab compares three sorting strategies and connects their implementation decisions to measured complexity. Insertion sort maintains a sorted prefix and is efficient on nearly sorted data but can require quadratic shifting on disordered data. Merge sort recursively divides the array and merges sorted halves, giving approximately `O(n log n)`. Basic quicksort partitions around a pivot, but choosing the first element can create unbalanced partitions on sorted input. We improve it by shuffling once, selecting a median-of-three pivot, and using insertion sort below an experimentally chosen cutoff. We verify every version with `Bench.java`, compare timing tables, explain the complexity changes, and submit the improved code, benchmark output, and completed answers."

Defense technique

When answering any code question, use this four-part pattern:

1. **Name the purpose:** say what the line or method is intended to do.

2. **Trace a small example:** show the indexes and values.
3. **Explain why it is correct:** connect the operation to the algorithm's invariant.
4. **Mention an edge case:** consider an empty array, one value, duplicates, sorted data, or reverse-sorted data.

This approach demonstrates understanding rather than memorisation.

References

[1] Repository README: Lab 1: Sorting, Complexity
[2] Repository Quick.java
[3] Repository Merge.java
[4] Repository Bench.java
[5] Repository answers.txt